

实验 2：命题逻辑与 SAT 求解

姓名：李文杰
学号：10245101584

1. 实验主题

把软件工程约束转换为命题逻辑和 CNF，使用 **MiniSAT** 求解，并检查编码、模型和解空间是否符合原始需求。

2. 实验目标

- 使用 MiniSAT 求解布尔可满足性问题（Boolean satisfiability problem, **SAT**），区分可满足（SAT）与不可满足（UNSAT）结果。
- 把自然语言约束编码为**合取范式**（conjunctive normal form, CNF）和 DIMACS CNF 文件格式，并维护变量—语义映射。
- 从 MiniSAT 模型恢复领域解，并独立检查每条原始约束。
- 使用**阻塞子句**（blocking clause，即排除当前模型、迫使求解器寻找不同模型的子句）分析无解、多解与唯一解。
- 使用 C++ 函数检查具体赋值是否满足 CNF，并通过完整比较有限变量上的模型集合判断两份编码是否等价。

允许使用 LLM 协助候选编码和脚本编写，但所有模型、反例和等价性结论都必须通过 MiniSAT 实际运行或完整赋值检查独立核验。

3. 工具与参考资料

- MiniSAT 官方代码库
- SAT Competition：DIMACS CNF 说明
- 可选：任意脚本语言，用于生成 CNF、解析模型和枚举多个解。

4. 最小环境检查（Smoke Test）

4.1 MiniSAT

- 保存以下内容为 minisat_smoke.cnf：

```
report.md • minisat_smoke.cnf X
minisat_smoke.cnf
1 c x1 must be true
2 p cnf 1 1
3 1 0
```

- 运行：

1 power shell运行报错：

```
PS D:\ME\FormalMethods\lab2> minisat minisat_smoke.cnf smoke_result.txt
minisat : 无法将“minisat”项识别为 cmdlet、函数、脚本文件或可运行程序的名称。请检查名称的拼写，如果包括路径，请确保路径正确，然后再试一次。
所在位置 行:1 字符: 1
+ minisat minisat_smoke.cnf smoke_result.txt
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (minisat:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
```

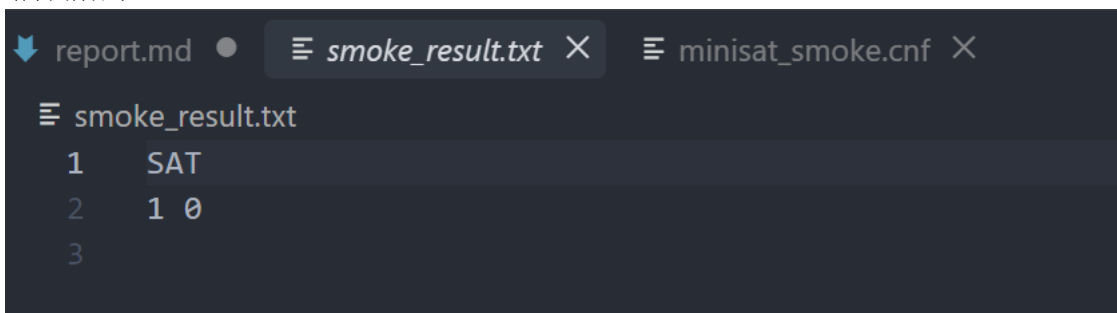
2 在linux环境下运行：

```
wsl: 检测到 localhost 代理配置, 但未镜像到 WSL。NAT 模式下的 WSL 不支持 localhost 代理。
jie@Saviour:~$ minisat
Command 'minisat' not found, but can be installed with:
sudo apt install minisat
jie@Saviour:~$ sudo apt install minisat
[sudo] password for jie:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  minisat
0 upgraded, 1 newly installed, 0 to remove and 83 not upgraded.
Need to get 88.1 kB of archives.
After this operation, 408 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 minisat amd64 1:2.2.1-8build1 [88.1 kB]
Fetched 88.1 kB in 3s (27.4 kB/s)
Selecting previously unselected package minisat.
(Reading database ... 50714 files and directories currently installed.)
Preparing to unpack .../minisat_1%3a2.2.1-8build1_amd64.deb ...
Unpacking minisat (1:2.2.1-8build1) ...
Setting up minisat (1:2.2.1-8build1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.9) ...
jie@Saviour:~$ cd /mnt/d/ME/FormalMethods/lab2
```

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat minisat_smoke.cnf smoke_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
| Number of variables:      1 |
| Number of clauses:       0 |
| Parse time:              0.00 s |
| Simplification time:     0.00 s |
===== [ Search Statistics ] =====
| Conflicts | ORIGINAL | LEARNNT | Progress |
| Vars  Clauses Literals | Limit  Clauses Lit/Cl |
=====
restarts      : 1
conflicts     : 0          (-nan /sec)
decisions     : 1          (0.00 % random) (inf /sec)
propagations  : 1          (inf /sec)
conflict literals : 0          (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0 s

SATISFIABLE
```

结果文件如下：



```
report.md  smoke_result.txt  minisat_smoke.cnf
smoke_result.txt
1 SAT
2 1 0
3
```

- 现在把唯一子句改为 -1 0 再运行，这次也得到 SAT，但模型中变量 1 为假：

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat minisat_smoke.cnf smoke_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:          1
| Number of clauses:           0
| Parse time:                  0.00 s
| Simplification time:         0.00 s
|
|===== [ Search Statistics ] =====
| Conflicts |          ORIGINAL          |          LEARNT          | Progress |
|           |      Vars  Clauses Literals  |      Limit  Clauses Lit/Cl |           |
|===== [ Search Statistics ] =====
restarts      : 1
conflicts     : 0              (0 /sec)
decisions     : 1              (0.00 % random) (237 /sec)
propagations  : 1              (237 /sec)
conflict literals : 0          (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.004226 s

SATISFIABLE
```

```
report.md • minisat_smoke.cnf × smoke_result.txt ×
smoke_result.txt
1 SAT
2 -1 0
3
```

5. 实验任务

任务 1：DIMACS 读写与结果解释

任务摘要： 读懂一个软件运行模式配置问题的 DIMACS 编码，通过修改约束构造不同版本，并从模型或约束传播过程解释求解结果。某软件可以采用服务器模式或本地模式，并根据运行模式选择数据库或文件存储后端。

定义变量如下：

变量	编号	为真时的含义
S	1	启用服务器模式
L	2	启用本地模式
D	3	启用数据库后端
F	4	启用文件存储后端
P	5	启用持久化功能

配置规则如下：

- 1. 服务器模式和本地模式至少启用一种。
- 2. 服务器模式要求数据库后端。
- 3. 本地模式要求文件存储后端。
- 4. 数据库后端要求启用持久化功能。
- 5. 文件存储后端要求启用持久化功能。
- 6. 启用持久化功能时，至少启用一种存储后端。
- 7. 数据库后端与文件存储后端不能同时启用。
- 8. 当前产品版本要求启用持久化功能。

保存以下内容为 base.cnf:

```
report.md ● base.cnf ●
base.cnf
1  c S=1, L=2, D=3, F=4, P=5
2  p cnf 5 8
3  1 2 0
4  -1 3 0
5  -2 4 0
6  -3 5 0
7  -4 5 0
8  -5 3 4 0
9  -3 -4 0
10 5 0
```

1.1 把八条业务规则分别写成命题逻辑公式，并建立“规则编号—逻辑公式—DIMACS 子句”对照表；同时检查 base.cnf 的文件头、变量编号、子句数量和结尾的 0。

- 文件的第一行是注释，规定编号：c S=1, L=2, D=3, F=4, P=5。每个变量只有 true 与 false 两种取值，
- 第二行p cnf 5 8则表示这个问题一共有5个变量，8条子句。下面的每一行都是一条必须满足的要求；
- CNF 的规则是：同一行中的文字用“或”连接，各行之间用“且”连接。 负号表示“非”，0 表示该子句结束。

规则编号	逻辑公式	DIMACS子句	规则含义
1	$S \vee L$	1 2 0	服务器模式和本地模式至少启用一种
2	$S \rightarrow D$	-1 3 0	服务器模式要求数据库后端
3	$L \rightarrow F$	-2 4 0	本地模式要求文件存储后端
4	$D \rightarrow P$	-3 5 0	数据库后端要求启用持久化功能
5	$F \rightarrow P$	-4 5 0	文件存储后端要求启用持久化功能
6	$P \rightarrow (D \vee F)$	-5 3 4 0	启用持久化功能时，至少启用一种存储后端
7	$\neg(D \wedge F)$	-3 -4 0	数据库后端与文件存储后端不能同时启用
8	P	5 0	当前产品版本要求启用持久化功能

- 持久化：把数据保存下来，程序退出后数据仍然保留。
- $\neg S \vee D$ 等价于 $S \rightarrow D$
- $\neg P \vee D \vee F$ 等价于 $\neg P \vee (D \vee F)$ 等价于 $P \rightarrow (D \vee F)$
- $\neg D \vee \neg F$ 等价于 $\neg(D \wedge F)$

1.2 运行 base.cnf，解读 MiniSAT 的终端输出和结果文件：说明求解状态表示什么，解释模型中正、负变量编号的含义，把模型还原为软件配置，并逐条检查该配置是否满足八条业务规则。

终端输出：

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat base.cnf base_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:      5
| Number of clauses:       7
| Parse time:              0.00 s
| Eliminated clauses:      0.00 Mb
| Simplification time:     0.00 s
|
|===== [ Search Statistics ]=====
| Conflicts | ORIGINAL | LEARNT | Progress |
|           | Vars  Clauses Literals | Limit  Clauses Lit/Cl |
|=====
restarts      : 1
conflicts     : 0          (0 /sec)
decisions     : 1          (0.00 % random) (463 /sec)
propagations  : 1          (463 /sec)
conflict literals : 0      (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.002161 s

SATISFIABLE
```

problem statistics:

结果文件:

```
report.md • base_result.txt X
base_result.txt
1 SAT
2 1 -2 3 -4 5 0
3
```

SAT表示存在满足全部约束的赋值，正数代表变量为真，负数则为假，结尾的0是结束标记

S=1, L=-2, D=3, F=-4, P=5 表示启用服务器模式，关闭本地模式，启用数据库后端，关闭文件存储后端，启用持久化功能。即：启用服务器模式、数据库后端和持久化功能，关闭本地模式和文件存储后端。

代入一下 base.cnf ，也确实满足八条业务规则：

规则编号	DIMACS子句	使子句为真的关键变量
1	1 2 0	1
2	-1 3 0	3
3	-2 4 0	-2
4	-3 5 0	5
5	-4 5 0	-4, 5
6	-5 3 4 0	3
7	-3 -4 0	-4
8	5 0	5

1.3 分别构造并运行两个变体：在基础文件 base.cnf 中增加单元子句 1 0 得到 server_only.cnf ，表示必须启用服务器模式；把基础文件最后的 5 0 改为 -5 0 得到 no_persistence.cnf ，表示禁止持久化。正确更新文件头，并比较三个版本的求解状态和模型变化。

- server_only.cnf:

```

report.md 9+  no_persistence.cnf  server_only.cnf
server_only.cnf
1  c S=1, L=2, D=3, F=4, P=5
2  p cnf 5 9
3  1 2 0
4  -1 3 0
5  -2 4 0
6  -3 5 0
7  -4 5 0
8  -5 3 4 0
9  -3 -4 0
10 5 0
11 1 0

```

运行结果如下:

```

jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat server_only.cnf server_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:          5
| Number of clauses:           7
| Parse time:                  0.00 s
| Simplification time:         0.00 s
|
|===== [ Search Statistics ] =====
| Conflicts | ORIGINAL | LEARNT | Progress |
|           | Vars  Clauses Literals | Limit  Clauses Lit/Cl |
|=====
restarts      : 1
conflicts     : 0          (0 /sec)
decisions     : 1          (0.00 % random) (272 /sec)
propagations  : 5          (1359 /sec)
conflict literals : 0      (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.003678 s

SATISFIABLE

jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat server_result.txt
SAT
1 -2 3 -4 5 0

```

- no_persistence.cnf:

```
report.md 9+ no_persistence.cnf server_only.cnf
no_persistence.cnf
1 c S=1, L=2, D=3, F=4, P=5
2 p cnf 5 8
3 1 2 0
4 -1 3 0
5 -2 4 0
6 -3 5 0
7 -4 5 0
8 -5 3 4 0
9 -3 -4 0
10 -5 0
```

运行结果如下：

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat no_persistence.cnf no_persistence_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:          5
| Number of clauses:           7
| Parse time:                   0.00 s
| Simplification time:          0.00 s
|
=====
Solved by simplification
restarts      : 0
conflicts    : 0          (0 /sec)
decisions    : 0          (-nan % random) (0 /sec)
propagations : 4          (1090 /sec)
conflict literals : 0      (-nan % deleted)
Memory used  : 11.00 MB
CPU time     : 0.00367 s

UNSATISFIABLE

jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat no_persistence_result.txt
UNSAT
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$
```

三个版本的求解状态和模型变化：

版本	变化	求解状态	输出模型	对应的配置
base.cnf	原始八条规则	SAT	1 -2 3 -4 5 0	启用服务器、数据库和持久化； 关闭本地模式和文件存储
server_only.cnf	新增 1 0，必须启用服务器模式	SAT	1 -2 3 -4 5 0	与基础版本输出的配置相同
no_persistence.cnf	将 5 0 替换为 -5 0， 禁止持久化	UNSAT	无满足模型	不存在满足全部约束的配置

因为base版本找到的模型满足server_only版本新增的约束1 0，所以二者输出模型一致。
但输出相同不表示约束没有作用：base版本允许本地模式方案，服务器版本将其排除了，因此合法配置的范围缩小了。

对于no_persistence版本：求解状态由SAT变为UNSAT。
禁止持久化后，数据库和文件存储都不能启用，进而导致服务器模式和本地模式都不能启用，而规定要求至少启用一种运行模式，所以发生了冲突。
因此，该版本没有满足模型。

1.4. 对 no_persistence.cn ，按照“持久化—存储后端—运行模式”的顺序解释为什么约束最终发生冲突，并列出一条冲突链实际使用的业务规则。

- 原因：
在禁用持久化之后，由于规则4、5（数据库后端/文件存储后端要求启用持久化功能）导致数据库/文件这两种存储后端无法启用，进一步根据规则2、3（服务器模式要求数据库后端/本地模式要求文件存储后端）导致服务器和本地这两种运行模式都不能启用，这进一步违反了规则1（服务器模式和本地模式至少启用一种）。
- 冲突链：
 $\neg P \rightarrow$
规则④ $\neg D \rightarrow$ 规则② $\neg S$
规则⑤ $\neg F \rightarrow$ 规则③ $\neg L$
 $\rightarrow$ 与规则① $S \vee L$ 冲突
实际使用的业务规则: 规则①、②、③、④、⑤。

求解结果为什么必须结合变量语义和约束关系解释：

MiniSAT 输出 SAT/UNSAT 和正负编号,但是只有结合变量对照表才能知道其代表的具体含义和满足的业务规则：

- SAT 意味着存在满足所有约束的取值，但具体启用了哪些功能要靠变量映射才能确定；
- UNSAT 意味着存在冲突的约束关系，但是冲突来自哪几条规则也要顺着约束链倒推。

如果不结合变量语义和约束关系，求解结果就只是无意义的数字，没有具体的作用。

任务 2：诚实者与说谎者建模

任务摘要： 把人物陈述转换为命题逻辑，判断问题是无解、唯一解还是多解，并解释判断依据。

场景如下：

- Alice 说“Bob 与 David 身份不同”；
- Bob 说“Alice 或 David 中至少一人诚实”；
- Carol 说“Alice 是说谎者”；
- David 说“Alice 或 Bob 中至少一人诚实”；
- Eve 说“Carol 是诚实者”。
- 另有“至少两名诚实者、至少两名说谎者”。

2.1. 明确区分“某人诚实”和“某人的陈述为真”，用等价关系表达身份规则，再转换为 CNF。

2.1.1 建模：

- 令A,B,C,D,E分别表示： Alice, Bob, Carol, David, Eve是诚实者。
- 同样地， $\neg A, \neg B, \neg C, \neg D, \neg E$ 分别表示： Alice, Bob, Carol, David, Eve是说谎者。
- 令 T_A, T_B, T_C, T_D, T_E 分别表示： Alice, Bob, Carol, David, Eve的陈述为真，反之则为假。

2.1.2 身份规则：

身份规则： 某人诚实当且仅当其陈述为真。所以有： $A \leftrightarrow T_A, B \leftrightarrow T_B, C \leftrightarrow T_C, D \leftrightarrow T_D, E \leftrightarrow T_E$ 。

代入陈述内容可得：

人物	陈述	陈述的逻辑表达式
Alice	Bob 与 David 身份不同	$A \leftrightarrow (B \wedge \neg D) \vee (\neg B \wedge D)$
Bob	Alice 或 David 至少一人诚实	$B \leftrightarrow A \vee D$
Carol	Alice 是说谎者	$C \leftrightarrow \neg A$
David	Alice 或 Bob 至少一人诚实	$D \leftrightarrow A \vee B$
Eve	Carol 是诚实者	$E \leftrightarrow C$

2.1.3 CNF：

①身份与陈述约束的 CNF：

Alice:
令 $Q = (B \wedge \neg D) \vee (\neg B \wedge D)$
 $\equiv (B \vee \neg B) \wedge (B \vee D) \wedge (\neg D \vee \neg B) \wedge (\neg D \vee D)$
 $\equiv (B \vee D) \wedge (\neg D \vee \neg B),$
 $\neg Q$
 $\equiv \neg((B \wedge \neg D) \vee (\neg B \wedge D))$

$$\begin{aligned} &\equiv \neg(B \wedge \neg D) \wedge \neg(\neg B \wedge D) \\ &\equiv (\neg B \vee D) \wedge (B \vee \neg D), \end{aligned}$$

$$\begin{aligned} A &\leftrightarrow (B \wedge \neg D) \vee (\neg B \wedge D) \\ &\equiv (A \rightarrow Q) \wedge (Q \rightarrow A) \\ &\equiv (\neg A \vee Q) \wedge (\neg Q \vee A) \\ &\equiv (\neg A \vee ((B \vee D) \wedge (\neg B \vee \neg D))) \wedge (A \vee ((\neg B \vee D) \wedge (B \vee \neg D))) \\ &\equiv (\neg A \vee B \vee D) \wedge (\neg A \vee \neg B \vee \neg D) \wedge (A \vee \neg B \vee D) \wedge (A \vee B \vee \neg D). \end{aligned}$$

如果Alice诚实，那么Bob、David至少一人诚实；

如果Alice诚实，那么Bob、David至少一人说谎；

如果Alice说谎且Bob诚实，那么David一定诚实；

如果Alice说谎且David诚实，那么Bob一定诚实；

Bob:

$$\begin{aligned} B &\leftrightarrow A \vee D \\ &\equiv (B \rightarrow (A \vee D)) \wedge ((A \vee D) \rightarrow B) \\ &\equiv (B \rightarrow (A \vee D)) \wedge (\neg(A \vee D) \vee B) \\ &\equiv (\neg B \vee A \vee D) \wedge (B \vee \neg A) \wedge (B \vee \neg D). \end{aligned}$$

如果Bob诚实，那么Alice、David至少一人诚实

如果Bob说谎，那么Alice一定说谎

如果Bob说谎，那么David一定说谎

Carol:

$$\begin{aligned} C &\leftrightarrow \neg A \\ &\equiv (C \rightarrow \neg A) \wedge (\neg A \rightarrow C) \\ &\equiv (\neg C \vee \neg A) \wedge (A \vee C) \end{aligned}$$

如果Carol诚实，那么Alice一定说谎

如果Carol说谎，那么Alice一定诚实

David:

$$\begin{aligned} D &\leftrightarrow A \vee B \text{ 同Bob} \\ &\equiv (\neg D \vee A \vee B) \wedge (D \vee \neg A) \wedge (D \vee \neg B). \end{aligned}$$

如果David诚实，那么Alice、Bob至少一人诚实

如果David说谎，那么Alice一定说谎

如果David说谎，那么Bob一定说谎

Eve:

$$\begin{aligned} E &\leftrightarrow C \\ &\equiv (E \rightarrow C) \wedge (C \rightarrow E) \\ &\equiv (\neg E \vee C) \wedge (\neg C \vee E) \end{aligned}$$

如果Eve诚实，那么Carol一定诚实

如果Eve说谎，那么Carol一定说谎

②人数约束的 CNF：

至少两名诚实者：

$$(A \vee B \vee C \vee D) \wedge (A \vee B \vee C \vee E) \wedge (A \vee B \vee D \vee E) \wedge (A \vee C \vee D \vee E) \wedge (B \vee C \vee D \vee E)$$

一共5人，至少2人诚实，即至多3人说谎，所以任选4人一定有诚实者

至少两名说谎者：

$$(\neg A \vee \neg B \vee \neg C \vee \neg D) \wedge (\neg A \vee \neg B \vee \neg C \vee \neg E) \wedge (\neg A \vee \neg B \vee \neg D \vee \neg E) \wedge (\neg A \vee \neg C \vee \neg D \vee \neg E) \wedge (\neg B \vee \neg C \vee \neg D \vee \neg E)$$

一共5人，至少2人说谎，即至多3人诚实，所以任选4人至少有1位说谎者

2.2. 运行 MiniSAT。如果找到一个模型，加入排除该模型的阻塞子句并再次求解，根据两次求解结果判定原问题是唯一解还是多解；如果第一次就是 UNSAT，则判定原问题无解，并从人物陈述的逻辑关系中解释一组直接冲突。

由2.1可知，一共有24条子句。

其中： **Alice:** 4; **Bob:** 3; **Carol:** 2; **David:** 3; **Eve:** 2; **至少两名诚实者:** 5; **至少两名说谎者:** 5。

按 A=1,B=2,C=3,D=4,E=5 (负数表示是说谎者)编号，把上述子句转化为DIMACS子句，保存在 honesty.cnf 文件中：

c A=1, B=2, C=3, D=4, E=5
p cnf 5 24

c Alice
-1 2 4 0
-1 -2 -4 0
1 -2 4 0
1 2 -4 0

c Bob
1 -2 4 0
-1 2 0
2 -4 0

c Carol
-1 -3 0
1 3 0

c David
1 2 -4 0
-1 4 0
-2 4 0

c Eve
3 -5 0
-3 5 0

c At least two honest people
1 2 3 4 0
1 2 3 5 0
1 2 4 5 0
1 3 4 5 0
2 3 4 5 0

c At least two liars
-1 -2 -3 -4 0
-1 -2 -3 -5 0
-1 -2 -4 -5 0
-1 -3 -4 -5 0
-2 -3 -4 -5 0

- 运行miniSAT:

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat honesty.cnf honesty_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:          5
| Number of clauses:           24
| Parse time:                   0.00 s
| Simplification time:          0.00 s
|
|
===== [ Search Statistics ] =====
| Conflicts | ORIGINAL | LEARNT | Progress |
|           | Vars  Clauses Literals | Limit  Clauses Lit/Cl |
|=====|=====|=====|=====|
restarts      : 1
conflicts     : 0          (0 /sec)
decisions     : 1          (0.00 % random) (160 /sec)
propagations  : 5          (799 /sec)
conflict literals : 0      (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.006254 s

SATISFIABLE
```

SATISFIABLE

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat honesty_result.txt
SAT
-1 -2 3 -4 5 0
```

- 加入排除 -1 -2 3 -4 5 0 答案的阻塞子句（honesty_blocked.cnf）后再次求解：

把 p cnf 5 24 改为 p cnf 5 25，在最后加上一行 1 2 -3 4 -5 0。

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat honesty_blocked.cnf honesty_blocked_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:          5
| Number of clauses:           25
| Parse time:                   0.00 s
| Simplification time:          0.00 s
|
|
=====
Solved by simplification
restarts      : 0
conflicts     : 0          (0 /sec)
decisions     : 0          (-nan % random) (0 /sec)
propagations  : 4          (659 /sec)
conflict literals : 0      (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.006072 s

UNSATISFIABLE
```

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat honesty_blocked_result.txt
UNSAT
```

至此可以得到：本题有**唯一解**：Alice、Bob、David是说谎者，Carol、Eve是诚实者，对应模型为： -1 -2 3 -4 5 0，该配置满足五人的身份与陈述之间的等价关系，并且包含两名诚实者、三名说谎者，满足人数约束。

任务 3：自选场景的 SAT 建模

任务摘要：使用 LLM 为自选的软件工程场景生成 SAT 模型，整理自然语言需求与 CNF 子句的对应关系，再使用 MiniSAT 求解。

从软件配置、测试用例选择、课程冲突或资源分配中选择一个场景。最终模型应包含不少于 10 个有业务含义的布尔变量和 15 个 CNF 子句，并至少包含依赖、互斥、至少一个、至多一个和条件组合中的三类关系。最终需求应当至少允许一个合法方案。

3.1. 编写提示词，让 LLM 生成场景说明、变量含义、编号后的自然语言需求、对应的 CNF 子句，以及一份满足全部需求的候选方案。提示词中应写明上述规模和可满足性要求。保存你实际提供给 LLM 的完整提示词。

SelectClassPrompt.txt:

```
> 一、身份认定
你正在完成“命题逻辑与SAT求解”实验。
而你是精通软件工程、形式化方法和SAT建模方面的专家，你需要为“学生选课与排课”场景设计一个候选SAT模型（要注意课程冲突）。

二、场景设定
教务系统已经给定各门课程的上课时间了，学生需要从选课系统里选择满足培养方案要求和课程搭配要求的课程，并且保证所选课程没有时间冲突。

三、模型要求
1. 课程不少于10门，并且每门课都有对应的上课时间段（每周几的第几节次，周末不上课），每一门课程都对应一个布尔变量：变量为真表示选择该课，为假表示不选择；
2. 最终CNF子句不少于15个；
3. 至少覆盖下列三类关系，并且要表明类别：（e.g.后面是给的例子，可以参考，不是必须这么写）
   (依赖：e.g.若选择某实践课程，必须同时选择配套该实践课程的理论课
   (互斥：e.g.上课时段有所重叠的课程不能同时选择
   (至少一个：e.g.某个可替代的公共课程组中至少选择一门
   (至多一个：e.g.通识类课程至多选择一门
   (条件组合：e.g.同时选择课程A和课程B时，必须选择配套课程C；
4. 最终需求应当至少允许一个合法方案，该方案要符合现实实际逻辑；
5. 你要自己设定课程表，包含课程名称类型以及上课时间，是否修读等；
6. 已修读表示已经修读并通过。已通过课程不再选择，其选课变量固定为假，但未通过且本学期要求修读的必修课必须选择（是否已通过是预先给定的学生信息，不是本模型要决定的变量。）；
7. 以上要求必须严格遵守，不得遗漏。

四、输出内容
请按照下列顺序输出：
1. 场景说明
2. 变量含义：
课程与变量表：课程名称、课程类型、上课时间、是否已通过、变量符号、DIMACS编号，以及变量为真的含义等等
1. 编号后的自然语言需求，并标明约束类别
2. 完整DIMACS CNF内容，以及对应的变量数和子句数
3. 一份满足全部需求的选课候选方案，明确所有变量的真假，列出所选课程及其时间
五、注意事项
1. 不要添加未在自然语言需求中说明的限制。
2. 如果候选方案不满足要求，请先修正模型或方案。
3. 所有课程默认在同一教学周范围内开设。只有在同一周几且节次重叠时，才视为时间冲突；必须编码课程表中的全部冲突。
4. 不要选择有时间冲突的课程，也不要误把时间不冲突的课程当成冲突。
5. 候选方案需逐条代入自然语言需求并对CNF子句进行检查。
```

3.2. 整理并检查 LLM 生成的模型，修正不合理或不一致的内容，形成最终的“自然语言需求—CNF 子句”对照，并把候选方案逐条代入最终需求和 CNF 检查。

3.2.1 输出结果：

● 验证全部通过：候选方案满足全部18个子句；课程表冲突对恰好为（x7,x12）、(x13,x14)两对且已全部编码；候选方案无时间冲突；模型共有17个满足赋值。以下是完整报告。

学生选课与排课 SAT 模型（命题逻辑与SAT求解实验）

1. 场景说明

某高校计算机科学与技术专业2024级学生（本学期为第4学期）在教务系统中进行本学期选课。教务系统已给定全部开课课程的上课时间：周一至周五上课，每天分若干节次，每门课连续占用2个节次（如“周一第1-2节”），周六、周日不安排课程。教务系统预先给定该生的修读信息（这些是既定事实，不是本模型要决定的变量）：

- 已修读并通过（本学期不得再选）：《大学物理（一）》《线性代数》；

- 未通过且本学期培养方案要求必须修读的必修课：《离散数学》《数据结构》《操作系统》《计算机网络》《数据库原理》《大学英语（三）》《体育（乒乓球）》；

- 其余课程（专业选修、实践课、通识选修）由学生自行决定是否修读，但必须满足培养方案的课程搭配要求，且所选课程上课时间不得冲突。

建模方式：每门课程对应一个布尔变量，变量为真表示选择该课程，为假表示不选择；将全部选课要求编码为CNF子句，求一组满足全部子句的选课方案（SAT）。

2. 课程与变量表							
序号	课程名称	课程类型	上课时间	是否已通过（预先给定）	变量符号	DIMACS编号	变量为真的含义
1	离散数学	专业必修	周一 第1-2节	否（本学期必须修读）	x1	1	选择《离散数学》
2	数据结构	专业必修	周一 第3-4节	否（本学期必须修读）	x2	2	选择《数据结构》
3	操作系统	专业必修	周二 第1-2节	否（本学期必须修读）	x3	3	选择《操作系统》
4	计算机网络	专业必修	周二 第3-4节	否（本学期必须修读）	x4	4	选择《计算机网络》
5	编译原理	专业选修	周三 第1-2节	否（自由选择）	x5	5	选择《编译原理》
6	编译原理课程设计	实践课（配套理论课：编译原理）	周三 第3-4节	否（自由选择）	x6	6	选择《编译原理课程设计》
7	人工智能导论	专业选修	周三 第5-6节	否（自由选择）	x7	7	选择《人工智能导论》
8	数据库原理	专业必修	周四 第1-2节	否（本学期必须修读）	x8	8	选择《数据库原理》
9	软件工程	专业选修	周四 第3-4节	否（自由选择）	x9	9	选择《软件工程》
10	大学英语（三）	公共必修	周一 第5-6节	否（本学期必须修读）	x10	10	选择《大学英语（三）》
11	体育（乒乓球）	公共必修	周二 第5-6节	否（本学期必须修读）	x11	11	选择《体育（乒乓球）》
12	艺术鉴赏	通识选修	周三 第5-6节	否（自由选择）	x12	12	选择《艺术鉴赏》
13	音乐欣赏	通识选修	周四 第5-6节	否（自由选择）	x13	13	选择《音乐欣赏》
14	数据结构课程设计	实践课（配套理论课：数据结构）	周四 第5-6节	否（自由选择）	x14	14	选择《数据结构课程设计》
15	大学物理（一）	公共必修	周五 第1-2节	是（已通过，不再选修）	x15	15	选择该课程（由约束固定为假）
16	线性代数	专业基础必修	周五 第3-4节	是（已通过，不再选修）	x16	16	选择该课程（由约束固定为假）

共16门课程（不少于10门），全部安排在同一教学周范围内，且均在周一至周五（周末不上课）。

3. 自然语言需求（编号 + 约束类别）

编号	类别	自然语言需求	逻辑式	对应CNF子句
R1	已修读信息·固定赋值	《大学物理(一)》《线性代数》已修读并通过，本学期不得再选	$\neg x_{15}, \neg x_{16}$	(-15)、(-16)
R2	已修读信息·固定赋值	未通过且本学期要求修读的7门必修课必须选择	$x_1 \wedge x_2 \wedge x_3 \wedge x_4 \wedge x_8 \wedge x_{10} \wedge x_{11}$	(1)(2)(3)(4)(8)(10)(11)
R3	依赖	若选择《编译原理课程设计》（实践课），必须同时选择配套理论课《编译原理》	$x_6 \rightarrow x_5$	(-6 5)
R4	依赖	若选择《数据结构课程设计》（实践课），必须同时选择配套理论课《数据结构》	$x_{14} \rightarrow x_2$	(-14 2)
R5	互斥	《人工智能导论》与《艺术鉴赏》同在周三第5-6节，时间重叠，不能同时选择	$\neg(x_7 \wedge x_{12})$	(-7 -12)
R6	互斥	《音乐欣赏》与《数据结构课程设计》同在周四第5-6节，时间重叠，不能同时选择	$\neg(x_{13} \wedge x_{14})$	(-13 -14)
R7	至少一个	专业选修课组{编译原理、人工智能导论、软件工程}中至少选择一门	$x_5 \vee x_7 \vee x_9$	(5 7 9)
R8	至少一个	通识选修课组{艺术鉴赏、音乐欣赏}中至少选择一门	$x_{12} \vee x_{13}$	(12 13)
R9	至多一个	通识选修课至多选择一门	$\neg(x_{12} \wedge x_{13})$	(-12 -13)
R10	至多一个	实践课{编译原理课程设计、数据结构课程设计}至多选择一门（本学期实践课负担上限）	$\neg(x_6 \wedge x_{14})$	(-6 -14)
R11	条件组合	若同时选择《编译原理》与《人工智能导论》，则必须选择《软件工程》（培养方案"智能软件方向"课程包要求）	$(x_5 \wedge x_7) \rightarrow x_9$	(-5 -7 9)

时间冲突编码完整性说明：对课程表中16门课程两两比对"同一周几且节次重叠"，全部冲突对恰好为（x7,x12）与（x13,x14）两对，已分别编码为R5、R6，既无遗漏，也没有把时间不冲突的课程误判为冲突（经程序核对一致）。

4. 完整DIMACS CNF

```
c 学生选课与排课SAT模型
c 变量：1..16 对应 x1..x16；子句数：18
c R1-R2 已修读信息固定赋值(单元子句)
c R3-R4 依赖； R5-R6 互斥； R7-R8 至少一个； R9-R10 至多一个； R11 条件组合
p cnf 16 18
1 0
2 0
3 0
4 0
8 0
10 0
11 0
-15 0
-16 0
-6 5 0
-14 2 0
-7 -12 0
-13 -14 0
5 7 9 0
12 13 0
-12 -13 0
-6 -14 0
-5 -7 9 0
```

变量数：16；子句数：18（不少于15）。

5. 候选选课方案（满足全部需求）

5.1 全部变量赋值

x1=x2=x3=x4=x5=x6=x7=x8=x9=x10=x11=x13=真； x12=x14=x15=x16=假。

5.2 所选课程及时间（共12门）

课程	时间	课程	时间
离散数学	周一 第1-2节	操作系统	周二 第1-2节
数据结构	周一 第3-4节	计算机网络	周二 第3-4节
大学英语(三)	周一 第5-6节	体育(乒乓球)	周二 第5-6节
编译原理	周三 第1-2节	数据库原理	周四 第1-2节
编译原理课程设计	周三 第3-4节	软件工程	周四 第3-4节
人工智能导论	周三 第5-6节	音乐欣赏	周四 第5-6节

不选：艺术鉴赏、数据结构课程设计；已通过不再选：大学物理(一)、线性代数。

方案解读：该生按"智能软件方向"选修编译原理+人工智能导论+软件工程（满足R11），并配套编译原理课程设计（满足R3）；因实践课至多一门（R10），放弃数据结构课程设计，周四第5-6节改选音乐欣赏（满足R8、R9，且不与任何课程冲突）；全部课程时间不冲突，周五无课，符合现实排课逻辑。

5.3 需求逐条代入检查

需求	代入检查	结论
R1	$x_{15}=F$ 、 $x_{16}=F$ ，已通过课程均未选择	满足
R2	$x_1\sim x_4$ 、 x_8 、 x_{10} 、 x_{11} 全为真，7门必修课均已选择	满足
R3	$x_6=T$ 且 $x_5=T$ ，选了实践课且配套理论课已选	满足
R4	$x_{14}=F$ ，前件不成立，蕴含式为真（即便选择， $x_2=T$ 亦满足）	满足
R5	$x_7=T$ 、 $x_{12}=F$ ，未同时选择，周三5-6节无冲突	满足
R6	$x_{13}=T$ 、 $x_{14}=F$ ，未同时选择，周四5-6节无冲突	满足
R7	$x_5=x_7=x_9=T$ ，专业选修实际选了3门	满足
R8	$x_{13}=T$ ，通识选修选了《音乐欣赏》	满足
R9	$x_{12}=F$ 、 $x_{13}=T$ ，通识恰好选一门	满足
R10	$x_6=T$ 、 $x_{14}=F$ ，实践课恰好选一门	满足
R11	$x_5=T\wedge x_7=T$ ，要求 $x_9=T$ ，实际 $x_9=T$	满足

5.4 CNF子句逐条代入检查

#	子句	代入	值
1	(x1)	T	真
2	(x2)	T	真
3	(x3)	T	真
4	(x4)	T	真
5	(x8)	T	真
6	(x10)	T	真
7	(x11)	T	真
8	(¬x15)	¬F=T	真
9	(¬x16)	¬F=T	真
10	(¬x6∨x5)	F∨T=T	真
11	(¬x14∨x2)	T∨T=T	真
12	(¬x7∨¬x12)	F∨T=T	真
13	(¬x13∨¬x14)	F∨T=T	真
14	(x5∨x7∨x9)	T∨T∨T=T	真
15	(x12∨x13)	F∨T=T	真
16	(¬x12∨¬x13)	T∨F=T	真
17	(¬x6∨¬x14)	F∨T=T	真
18	(¬x5∨¬x7∨x9)	F∨F∨T=T	真

18个子句全部为真。

5.5 时间冲突检查

- 周一：离散数学(1-2)、数据结构(3-4)、大学英语(三)(5-6)——无重叠
- 周二：操作系统(1-2)、计算机网络(3-4)、体育(5-6)——无重叠
- 周三：编译原理(1-2)、编译原理课程设计(3-4)、人工智能导论(5-6)——无重叠
- 周四：数据库原理(1-2)、软件工程(3-4)、音乐欣赏(5-6)——无重叠
- 周五：无课

所选12门课程中不存在任何同一周几且节次重叠的课程对，时间全部合法。

总结：模型共16个变量、18个子句，覆盖依赖、互斥、至少一个、至多一个、条件组合五类约束；经程序穷举全部 $2^{16}=65536$ 种赋值验证，模型可满足（SAT），共有17个满足全部子句的赋值，上述候选方案为其中之一。

* Baked for 8m 6s

※ recap: Your course-selection SAT model report is complete and verified: 16 courses, 18 CNF clauses covering all five constraint categories, with a valid 12-course candidate solution checked clause by clause. Next step, if you want, I can save the model.cnf file for your solver. (disable recaps in /config)

3.2.2 整理与检查：

检查项目	当前模型	检查结果
变量数量不少于10	16个变量，各对应一门课程	满足
子句数量不少于15	18条子句，文件头为 p cnf 16 18	满足
至少三类关系	依赖、互斥、至少一个、至多一个、条件组合	满足
变量含义一致	真表示选择，假表示不选择；是否已通过是已知信息	一致
已通过课程不再选择	-15 0、 -16 0	正确
必修课程必须选择	x1、x2、x3、x4、x8、x10、x11 被固定为真	正确
时间冲突编码完整	x7与x12、x13与x14 两对冲突均已编码	正确

3.2.3 最终的“自然语言需求—CNF 子句”对照：

需求	自然语言要求	DIMACS 子句
R1	两门已通过课程不再选择	-15 0； -16 0
R2	七门本学期必修课必须选择	1 0； 2 0； 3 0； 4 0； 8 0； 10 0； 11 0
R3	编译原理课程设计依赖编译原理	-6 5 0
R4	数据结构课程设计依赖数据结构	-14 2 0
R5	人工智能导论与艺术鉴赏时间冲突	-7 -12 0
R6	音乐欣赏与数据结构课程设计时间冲突	-13 -14 0
R7	三门专业选修课至少选择一门	5 7 9 0
R8	两门通识选修课至少选择一门	12 13 0
R9	两门通识选修课至多选择一门	-12 -13 0
R10	两门实践课至多选择一门	-6 -14 0
R11	同选编译原理和人工智能导论时，必须选择软件工程	-5 -7 9 0

3.2.4 把候选方案逐条代入最终需求和CNF检查：

LLM给出的方案：

x1=x2=x3=x4=x5=x6=x7=x8=x9=x10=x11=x13=真； x12=x14=x15=x16=假。

带入3.2.3的表格：

需求	自然语言要求	DIMACS 子句	结果
R1	两门已通过课程不再选择	-15 0； -16 0	T, T
R2	七门本学期必修课必须选择	1 0； 2 0； 3 0； 4 0； 8 0； 10 0； 11 0	T; T; T; T; T; T; T;
R3	编译原理课程设计依赖编译原理	-6 5 0	$F \vee T = T$
R4	数据结构课程设计依赖数据结构	-14 2 0	$T \vee T = T$
R5	人工智能导论与艺术鉴赏时间冲突	-7 -12 0	$F \vee T = T$
R6	音乐欣赏与数据结构课程设计时间冲突	-13 -14 0	$F \vee T = T$
R7	三门专业选修课至少选择一门	5 7 9 0	$T \vee T \vee T = T$
R8	两门通识选修课至少选择一门	12 13 0	$F \vee T = T$
R9	两门通识选修课至多选择一门	-12 -13 0	$T \vee F = T$
R10	两门实践课至多选择一门	-6 -14 0	$F \vee T = T$
R11	同选编译原理和人工智能导论时，必须选择软件工程	-5 -7 9 0	$F \vee F \vee T = T$

3.3. 将最终 CNF 写成 DIMACS 文件并使用 MiniSAT 求解，把求得的模型还原为业务方案并逐条检查需求。若MiniSAT 报告 UNSAT，应检查需求或编码并修正，直到最终提交的模型可满足。

3.3.1 将最终 CNF 写成 DIMACS 文件:

selection.cnf:

```
c 学生选课与排课SAT模型
c 变量: 1..16 对应 x1..x16
p cnf 16 18
1 0
2 0
3 0
4 0
8 0
10 0
11 0
-15 0
-16 0
-6 5 0
-14 2 0
-7 -12 0
-13 -14 0
5 7 9 0
12 13 0
-12 -13 0
-6 -14 0
-5 -7 9 0
```

3.3.2 使用 MiniSAT 求解:

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat selection.cnf selection_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:      16
| Number of clauses:       8
| Parse time:              0.00 s
| Eliminated clauses:      0.00 Mb
| Simplification time:     0.00 s
|
===== [ Search Statistics ] =====
| Conflicts |          ORIGINAL          |      LEARNT      | Progress |
|           | Vars  Clauses Literals |   Limit  Clauses Lit/Cl |           |
=====
restarts      : 1
conflicts     : 0          (-nan /sec)
decisions     : 1          (0.00 % random) (inf /sec)
propagations  : 9          (inf /sec)
conflict literals : 0      (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0 s

SATISFIABLE
```

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat selection_result.txt
SAT
1 2 3 4 -5 -6 -7 8 9 10 11 12 -13 -14 -15 -16 0
```

3.3.3 把求得的模型还原为业务方案并逐条检查需求:

可以看到，这个结果不同于LLM给出的结果（1 2 3 4 5 6 7 8 9 10 11 -12 13 -14 -15 -16 0），这说明该模型不止唯一解。主要区别是：

项目	LLM 候选方案	MiniSat 运行结果
专业选修	编译原理、人工智能导论、软件工程	软件工程
实践课	编译原理课程设计	不选实践课
通识选修	音乐欣赏	艺术鉴赏
总课程数	12 门	9 门

现在我们把miniSAT的结果还原为业务方案并逐条检查：

需求	自然语言要求	实际方案检查	结果
R1	已通过课程不再选	x15、x16均为假	满足
R2	七门必修课必须选	x1、x2、x3、x4、x8、x10、x11均为真	满足
R3	编译实践依赖理论	x6为假，未选择该实践课，不触发要求	满足
R4	数据结构实践依赖理论	x14为假，不触发要求	满足
R5	人工智能与艺术鉴赏互斥	x7为假、x12为真	满足
R6	音乐欣赏与数据结构实践互斥	x13、x14均为假	满足
R7	专业选修至少一门	已选择x9	满足
R8	通识选修至少一门	已选择x12	满足
R9	通识选修至多一门	仅选择x12	满足
R10	实践课至多一门	实际选择零门	满足
R11	同选编译原理和人工智能时须选软件工程	x5、x7均为假，条件未触发	满足

综上，最终CNF能够正确表达需求。

任务 4（编程题）：实现 CNF 赋值检查器

任务摘要： 按照固定接口实现一个 C++ 函数，判断给定布尔赋值是否满足一组 CNF 子句，并观察一次单变量翻转怎样使具体子句失效。

- 请在 lab2_solution.cpp 中实现以下函数：

```
#include <vector>

bool satisfiesCnf(
    int variableCount,
    const std::vector<std::vector<int>>& clauses,
    const std::vector<int>& assignment) {
    // 在此处完成实现
}
```

接口约定：

- 变量编号为1到variableCount。
- assignment恰好包含variableCount个元素；assignment[i]表示变量i + 1的值，只可能为0或1。
- 正整数文字表示对应变量为真，负整数文字表示对应变量为假。
- 每个子句至少包含一个文字，且文字的绝对值不超过variableCount。
- 只有每个子句都至少有一个文字为真时，函数才返回true。
- 函数不得调用MiniSAT，不得定义main，不得进行标准输入输出；评分程序将使用C++17编译并直接调用该函数。

4.1. 实现satisfiesCnf，正确处理正文字、负文字、单文字子句和多个子句。

lab2_solution.cpp：

```
#include <vector>
bool satisfiesCnf(
    int variableCount, //变量总数
    const std::vector<std::vector<int>>& clauses, //子句
    const std::vector<int>& assignment) { //保存变量的真假值
    //遍历所有的子句
    for(const auto& clause : clauses){ //clause: 本轮循环取出的一个子句; auto: 让编译器自动推断类型
        bool clauseSatisfied = false; //假设这个子句目前是不满足的
        //遍历当前子句中的所有文字
        for(int literal : clause){ //literal是当前取出的一个整数文字。
            int variable = literal > 0 ? literal : -literal; //取出文字对应的变量编号
            bool value = assignment[variable - 1] == 1; //读取该变量的真假值
            bool literalSatisfied = literal > 0 ? value : !value; //根据正负号计算文字的真假
            if(literalSatisfied){ //判断这个布尔变量是否为真
                clauseSatisfied = true;
                break;
            }
        }
        if(!clauseSatisfied){
            return false;
        }
    }
    return true;
}
```

4.2. 自行设计至少六个测试，其中应包含满足赋值和不满足赋值。

编号	variableCount	clauses	assignment	预期结果	原因
1	1	{1}	{1}	true	x_1 为真
2	1	{1}	{0}	false	x_1 为假
3	1	{-1}	{0}	true	x_1 为假，所以 $\neg x_1$ 为真
4	1	{-1}	{1}	false	x_1 为真，所以 $\neg x_1$ 为假
5	3	{1,-2},{-1,3},{2,3}	{1,0,1}	true	三个子句都满足
6	3	{1,-2},{-1,3},{2,3}	{1,0,0}	false	第二、第三个子句均不满足

新增 test_42.cpp

- 代码如下：

```

#include <iostream>
#include <vector>
#include "lab2_solution.cpp"

int main() {
    // 每个测试记录：变量数、子句、赋值和预期结果。
    struct TestCase {
        int variableCount;
        std::vector<std::vector<int>> clauses;
        std::vector<int> assignment;
        bool expected;
    };

    std::vector<TestCase> tests = {
        {1, {{1}}, {1}, true},
        {1, {{1}}, {0}, false},
        {1, {{-1}}, {0}, true},
        {1, {{-1}}, {1}, false},
        {3, {{1, -2}, {-1, 3}, {2, 3}}, {1, 0, 1}, true},
        {3, {{1, -2}, {-1, 3}, {2, 3}}, {1, 0, 0}, false}
    };

    int passed = 0;
    std::cout << std::boolalpha;

    for (int i = 0; i < static_cast<int>(tests.size()); ++i) {
        const auto& test = tests[i];

        // 实际调用函数，得到检查结果。
        bool actual = satisfiesCnf(
            test.variableCount,
            test.clauses,
            test.assignment
        );

        bool match = actual == test.expected;

        if (match) {
            ++passed;
        }

        std::cout << "Test " << i + 1
            << ": expected=" << test.expected
            << ", actual=" << actual
            << ", " << (match ? "PASS" : "FAIL")
            << '\n';
    }

    std::cout << "Passed: " << passed
        << "/" << tests.size() << '\n';

    return passed == static_cast<int>(tests.size()) ? 0 : 1;
}

```

- 检查结果如下：

```

Test 1: expected=true, actual=true, PASS
Test 2: expected=false, actual=false, PASS
Test 3: expected=true, actual=true, PASS
Test 4: expected=false, actual=false, PASS
Test 5: expected=true, actual=true, PASS
Test 6: expected=false, actual=false, PASS
Passed: 6/6

```

4.3. 使用固定 CNF $\{\{1, 2\}, \{-1, 3\}, \{-2, -3\}\}$ 和赋值 $x1=true, x2=false, x3=true$ 检查函数，确认其返回 true。随后只把 $x3$ 改为 false，再次运行函数，并指出哪个子句因此变为不满足。

用 test_fixed.cpp 来检查函数

```
#include <iostream>
#include <vector>
#include "lab2_solution.cpp"

int main() {
    std::vector<std::vector<int>> clauses = {
        {1, 2}, {-1, 3}, {-2, -3}
    };

    std::cout << std::boolalpha;
    bool allMatch = true;

    // 分别检查 x3=true 和 x3=false, x1、x2 保持不变。
    for (int x3 : {1, 0}) {
        std::vector<int> assignment = {1, 0, x3};

        // 待验证函数的计算结果。
        bool actual = satisfiesCnf(3, clauses, assignment);

        // 独立按原始逻辑公式计算，不调用 satisfiesCnf。
        bool x1Value = assignment[0] == 1;
        bool x2Value = assignment[1] == 1;
        bool x3Value = assignment[2] == 1;

        bool c1 = x1Value || x2Value;
        bool c2 = !x1Value || x3Value;
        bool c3 = !x2Value || !x3Value;
        bool direct = c1 && c2 && c3;

        bool match = actual == direct;
        allMatch = allMatch && match;

        std::cout << "Assignment: "
            << x1Value << ", "
            << x2Value << ", "
            << x3Value << '\n';

        std::cout << "Clause 1: " << c1 << '\n';
        std::cout << "Clause 2: " << c2 << '\n';
        std::cout << "Clause 3: " << c3 << '\n';

        std::cout << "Direct evaluation: " << direct << '\n';
        std::cout << "satisfiesCnf: " << actual << '\n';
        std::cout << "Match: " << match << "\n\n";
    }

    return allMatch ? 0 : 1;
}
```

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ g++ -std=c++17 test_fixed.cpp -o test_fixed
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ ./test_fixed
Assignment: true, false, true
Clause 1: true
Clause 2: true
Clause 3: true
Direct evaluation: true
satisfiesCnf: true
Match: true

Assignment: true, false, false
Clause 1: true
Clause 2: false
Clause 3: true
Direct evaluation: false
satisfiesCnf: false
Match: true
```

根据输出结果，我们可以看到：

CNF	检查项	翻转前 {1,0,1}	翻转后 {1,0,0}
{1,2}	$x_1 \vee x_2$	true	true
{-1,3}	$\neg x_1 \vee x_3$	true	false
{-2,-3}	$\neg x_2 \vee \neg x_3$	true	true
\	原始公式直接计算	true	false
\	satisfiesCnf 返回值	true	false
\	两种计算结果是否一致	是	是

可以看到，在 x_3 由 true 变为 false 后，{-1, 3} (-1 3 0) 因此变得不再满足

对CNF中“所有子句均满足”含义撰写的结论：

对CNF来说，“所有子句均满足”是判定整个命题公式为真的**充要**条件，因为CNF是各子句的合取（ $\wedge$ ），所以只有所有的子句均为真才可以得到整个命题为真的结论。而对于子句，因为其是文字的析取（ $\vee$ ），只要子句内部其中一个文字为真，这个子句就是真。

任务 5：使用阻塞子句寻找多个解

任务摘要： 每找到一个解，就增加一条排除该解的阻塞子句，继续求解，从而枚举同一组业务规则下的多个不同方案。

阻塞子句（blocking clause）用于排除求解器刚刚找到的完整赋值。例如当前模型为 $x_1=\text{true}, x_2=\text{false}, x_3=\text{true}$ ，可加入 $(\neg x_1 \text{ OR } x_2 \text{ OR } \neg x_3)$ ；这条子句只排除该模型，不直接指定下一个答案。

本任务使用一个固定的测试策略选择问题，不依赖任务 3 的自选模型。
定义变量如下：

变量	编号	为真时的含义
U	1	执行单元测试
I	2	执行集成测试
S	3	执行静态分析
C	4	收集覆盖率

规则如下：
单元测试和集成测试至少执行一种；
静态分析要求执行单元测试；
覆盖率收集要求执行集成测试；
静态分析和覆盖率收集不能同时启用。
保存为 test_strategy.cnf：

```
c U=1, I=2, S=3, C=4
p cnf 4 4
1 2 0
-3 1 0
-4 2 0
-3 -4 0
```

5.1. 运行原始 CNF，把第一个模型还原为测试策略并逐条检查四条规则。

5.1.1 运行 test_strategy.cnf：

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat test_strategy.cnf strategy_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:          4
| Number of clauses:           4
| Parse time:                  0.00 s
| Eliminated clauses:          0.00 Mb
| Simplification time:         0.00 s
|
===== [ Search Statistics ] =====
| Conflicts |          ORIGINAL          |          LEARNNT          | Progress |
|           | Vars  Clauses Literals |   Limit  Clauses Lit/Cl   |          |
=====
restarts      : 1
conflicts     : 0              (0 /sec)
decisions     : 1              (0.00 % random) (211 /sec)
propagations  : 0              (0 /sec)
conflict literals : 0          (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.004733 s

SATISFIABLE
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat strategy_result.txt
SAT
1 -2 -3 -4 0
```

5.1.2 把第一个模型还原为测试策略并逐条检查四条规则:

首个模型为 1 -2 -3 -4 0，表示只执行单元测试，即 $U = T, I = F, S = F, C = F$

规则	DIMACS子句	逻辑公式	测试策略	代入结果
1	1 2 0	$U \vee I$	单元测试和集成测试至少要进行一种	T
2	-3 1 0	$\neg S \vee U$	静态分析要求执行单元测试	T
3	-4 2 0	$\neg C \vee I$	收集覆盖率要求执行集成测试	T
4	-3 -4 0	$\neg S \vee \neg C$	静态分析和收集覆盖率不能同时执行	T

注释: $\neg S \vee U$ 即 $S \rightarrow U$ ， $\neg C \vee I$ 即 $C \rightarrow I$ ， $\neg S \vee \neg C$ 即 $\neg(S \wedge C)$

5.2. 根据每次得到的完整模型构造阻塞子句，重复运行 MiniSAT，直到结果变为 UNSAT。列出全部不同测试策略和每次加入的阻塞子句。

5.2.1 根据每次得到的完整模型构造阻塞子句，重复运行 MiniSAT，直到结果变为 UNSAT：

PS：将阻塞子句加入最后一行的时候要同步更新 p cnf 4 4 ,每加一次都应使最右侧的数字加一。
我在这里忘记加了所以有警告 WARNING!DIMACS header mismatch: wrong number of clauses.，不过 MiniSAT 仍读取了新增子句（截图中的 number of clauses 是+1了的），所以枚举结果也没有因此失效，我也就没有重做了。

第一次运行的结果：

```

jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat test_strategy.cnf strategy_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:          4
| Number of clauses:           4
| Parse time:                   0.00 s
| Eliminated clauses:          0.00 Mb
| Simplification time:          0.00 s
|
===== [ Search Statistics ] =====
| Conflicts | ORIGINAL | LEARNT | Progress |
|           | Vars  Clauses Literals | Limit  Clauses Lit/C1 |
=====
restarts      : 1
conflicts     : 0                (0 /sec)
decisions     : 1                (0.00 % random) (211 /sec)
propagations  : 0                (0 /sec)
conflict literals : 0            (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.004733 s

SATISFIABLE
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat strategy_result.txt
SAT
1 -2 -3 -4 0

```

构造阻塞子句：-1 2 3 4 0，加入 test_strategy.cnf 最后一行。

第二次运行的结果：

```

jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat test_strategy.cnf strategy_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| WARNING! DIMACS header mismatch: wrong number of clauses.
| Number of variables:          4
| Number of clauses:           5
| Parse time:                   0.00 s
| Eliminated clauses:          0.00 Mb
| Simplification time:          0.00 s
|
===== [ Search Statistics ] =====
| Conflicts | ORIGINAL | LEARNT | Progress |
|           | Vars  Clauses Literals | Limit  Clauses Lit/C1 |
=====
restarts      : 1
conflicts     : 0                (0 /sec)
decisions     : 1                (0.00 % random) (245 /sec)
propagations  : 0                (0 /sec)
conflict literals : 0            (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.004084 s

SATISFIABLE
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat strategy_result.txt
SAT
-1 2 -3 -4 0

```

构造阻塞子句： 1 -2 3 4 0 ，加入 test_strategy.cnf 最后一行。

第三次运行的结果：

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat test_strategy.cnf strategy_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| WARNING! DIMACS header mismatch: wrong number of clauses.
| Number of variables:          4
| Number of clauses:           6
| Parse time:                  0.00 s
| Eliminated clauses:          0.00 Mb
| Simplification time:         0.00 s
|
===== [ Search Statistics ] =====
| Conflicts | ORIGINAL | LEARNT | Progress |
|           | Vars  Clauses Literals | Limit  Clauses Lit/Cl |
=====
restarts      : 1
conflicts     : 0          (-nan /sec)
decisions     : 1          (0.00 % random) (inf /sec)
propagations  : 0          (-nan /sec)
conflict literals : 0          (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0 s

SATISFIABLE
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat strategy_result.txt
SAT
1 2 -3 -4 0
```

构造阻塞子句： -1 -2 3 4 0 ，加入 test_strategy.cnf 最后一行。

第四次运行的结果：

```

jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat test_strategy.cnf strategy_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
|
WARNING! DIMACS header mismatch: wrong number of clauses.
| Number of variables:      4
| Number of clauses:       7
| Parse time:              0.00 s
| Eliminated clauses:      0.00 Mb
| Simplification time:     0.00 s
|
|
===== [ Search Statistics ] =====
| Conflicts | ORIGINAL | LEARNT | Progress |
|           | Vars  Clauses Literals | Limit  Clauses Lit/Cl |
|=====|=====|=====|=====|
restarts      : 1
conflicts     : 0          (-nan /sec)
decisions     : 1          (0.00 % random) (inf /sec)
propagations  : 0          (-nan /sec)
conflict literals : 0          (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0 s

SATISFIABLE
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat strategy_result.txt
SAT
1 2 3 -4 0

```

构造阻塞子句：-1 -2 -3 4 0，加入 test_strategy.cnf 最后一行。

第五次运行的结果：

```

jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat test_strategy.cnf strategy_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
|
WARNING! DIMACS header mismatch: wrong number of clauses.
| Number of variables:      4
| Number of clauses:       8
| Parse time:              0.00 s
| Eliminated clauses:      0.00 Mb
| Simplification time:     0.00 s
|
|
===== [ Search Statistics ] =====
| Conflicts | ORIGINAL | LEARNT | Progress |
|           | Vars  Clauses Literals | Limit  Clauses Lit/Cl |
|=====|=====|=====|=====|
restarts      : 1
conflicts     : 0          (0 /sec)
decisions     : 1          (0.00 % random) (337 /sec)
propagations  : 0          (0 /sec)
conflict literals : 0          (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.002968 s

SATISFIABLE
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat strategy_result.txt
SAT
1 -2 3 -4 0

```

构造阻塞子句：-1 2 -3 4 0，加入 test_strategy.cnf 最后一行。

第六次运行的结果：

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat test_strategy.cnf strategy_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| WARNING! DIMACS header mismatch: wrong number of clauses.
| Number of variables:      4
| Number of clauses:       9
| Parse time:              0.00 s
| Eliminated clauses:      0.00 Mb
| Simplification time:     0.00 s
|
===== [ Search Statistics ] =====
| Conflicts |          ORIGINAL          |          LEARNT          | Progress |
|           |  Vars  Clauses Literals  |  Limit  Clauses Lit/Cl   |          |
=====
restarts      : 1
conflicts     : 0              (0 /sec)
decisions     : 1              (0.00 % random) (291 /sec)
propagations  : 3              (872 /sec)
conflict literals : 0          (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.003442 s

SATISFIABLE
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat strategy_result.txt
SAT
-1 2 -3 4 0
```

构造阻塞子句：1 -2 3 -4 0，加入 test_strategy.cnf 最后一行。

第七次运行的结果：

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat test_strategy.cnf strategy_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| WARNING! DIMACS header mismatch: wrong number of clauses.
| Number of variables:      4
| Number of clauses:      10
| Parse time:              0.00 s
| Simplification time:     0.00 s
|
===== [ Search Statistics ] =====
| Conflicts |          ORIGINAL          |          LEARNT          | Progress |
|           |  Vars  Clauses Literals  |  Limit  Clauses Lit/Cl   |          |
=====
restarts      : 1
conflicts     : 0              (0 /sec)
decisions     : 1              (0.00 % random) (355 /sec)
propagations  : 4              (1418 /sec)
conflict literals : 0          (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.00282 s

SATISFIABLE
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat strategy_result.txt
SAT
1 2 -3 4 0
```

构造阻塞子句： -1 -2 3 -4 0 ，加入 test_strategy.cnf 最后一行。

第八次运行的结果

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat test_strategy.cnf strategy_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
|
WARNING! DIMACS header mismatch: wrong number of clauses.
| Number of variables:      4
| Number of clauses:       11
| Parse time:              0.00 s
| Simplification time:     0.00 s
|
|
=====
Solved by simplification
restarts      : 0
conflicts    : 0              (0 /sec)
decisions    : 0              (-nan % random) (0 /sec)
propagations : 4              (1433 /sec)
conflict literals : 0          (-nan % deleted)
Memory used  : 11.00 MB
CPU time     : 0.002792 s

UNSATISFIABLE
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat strategy_result.txt
UNSAT
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$
```

这次的结果终于变成了 UNSAT 。

5.2.2 列出全部不同测试策略和每次加入的阻塞子句：

运行编号	运行结果	对应的测试策略	最后加入的阻塞子句
1	SAT , 1 -2 -3 -4 0	只执行单元测试	-1 2 3 4 0
2	SAT , -1 2 -3 -4 0	只执行集成测试	1 -2 3 4 0
3	SAT , 1 2 -3 -4 0	同时执行单元测试与集成测试	-1 -2 3 4 0
4	SAT , 1 2 3 -4 0	同时执行单元测试、集成测试与静态分析	-1 -2 -3 4 0
5	SAT , 1 -2 3 -4 0	同时执行单元测试与静态分析	-1 2 -3 4 0
6	SAT , -1 2 -3 4 0	执行集成测试的同时收集覆盖率	1 -2 3 -4 0
7	SAT , 1 2 -3 4 0	同时执行单元测试与集成测试并收集覆盖率	-1 -2 3 -4 0
8	UNSAT	无	无

5.3. 每得到一个模型，都使用任务 4 的 satisfiesCnf 检查原始四条子句，并确认新模型在四个业务变量上与此前模型不同。根据最终的 UNSAT 说明为什么已经没有遗漏其它完整赋值。

5.3.1每得到一个模型，都使用任务 4 的 satisfiesCnf 检查原始四条子句，并确认新模型在四个业务变量上与此前模型不同：

原始4条子句是 1 2 0 , -3 1 0 , -4 2 0 , -3 -4 0 ，所以 clauses={{1, 2}, {-3, 1},{-4, 2},{-3, -4}} ，
然后依次调用satisfiesCnf()，检查5.2中的7个模型 （ {1, 0, 0, 0},{0, 1, 0, 0},{1, 1, 0, 0},{1, 1, 1, 0},{1, 0, 1, 0},{0, 1, 0, 1},{1, 1, 0, 1} ）
并彼此进行比较，确认四个业务变量与此前不同。
代码如下：

```

#include <iostream>
#include <vector>
#include "lab2_solution.cpp"

int main() {
    // 只检查原始四条业务规则，不加入阻塞子句
    const std::vector<std::vector<int>> clauses = {
        {1, 2},
        {-3, 1},
        {-4, 2},
        {-3, -4}
    };
    // 每行依次表示U、I、S、C，1为真，0为假
    const std::vector<std::vector<int>> models = {
        {1, 0, 0, 0},
        {0, 1, 0, 0},
        {1, 1, 0, 0},
        {1, 1, 1, 0},
        {1, 0, 1, 0},
        {0, 1, 0, 1},
        {1, 1, 0, 1}
    };

    bool allPassed = true;
    //让bool类型以true/false的形式输出，而不是1/0
    std::cout << std::boolalpha;
    for (int i = 0; i < static_cast<int>(models.size()); i++){
        // 检查当前模型是否满足原始CNF
        bool satisfies = satisfiesCnf(4, clauses, models[i]);
        // 将当前模型与此前的每一个模型比较
        bool isNew = true;
        for (int j = 0; j < i; j++){
            if (models[i] == models[j]) {
                isNew = false;
                break;
            }
        }
        //
        std::cout << "Model " << i + 1 << ": ";
        for (int value : models[i]) {
            std::cout << value << ' ';
        }
        std::cout << "| satisfies=" << satisfies
            << " | new=" << isNew << '\n';
        //只要有一项不通过，整体就记为失败
        if (!satisfies || !isNew) {
            allPassed = false;
        }
    }
    std::cout << "All checks passed: " << allPassed << '\n';
    return allPassed ? 0 : 1;
}

```

运行结果如下：

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ g++ test_strategy_check.cpp -o test_strategy_check
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ ./test_strategy_check
Model 1: 1 0 0 0 | satisfies=true | new=true
Model 2: 0 1 0 0 | satisfies=true | new=true
Model 3: 1 1 0 0 | satisfies=true | new=true
Model 4: 1 1 1 0 | satisfies=true | new=true
Model 5: 1 0 1 0 | satisfies=true | new=true
Model 6: 0 1 0 1 | satisfies=true | new=true
Model 7: 1 1 0 1 | satisfies=true | new=true
All checks passed: true
```

5.3.2 根据最终的 UNSAT 说明为什么已经没有遗漏其它完整赋值：

首先，在前面5.3.1的验证中，运行结果显示，七个模型均满足原始CNF并且是互不相同的，这说明了5.2得到的7个模型也都是正确的。

在第七次将之前的结果全部剔除之后（通过阻塞子句：只排除刚刚找到的完整赋值，不会排除其他完整赋值），第八次得到 UNSAT 。

第八次得到 UNSAT 说明不再存在这样的赋值了：因为如果还有其他赋值的话，它应当与之前找到的七个模型都不同，仍能满足全部阻塞子句，结果应该返回 SAT，而第八次的结果是 UNSAT，所以不存在其它满足规则的完整赋值了，故而没有遗漏。

5.4. 增加业务规则“必须执行静态分析”，即加入单元子句 3 0，重新完整枚举。比较修改前后的方案数量，并解释哪些方案被新规则排除。

5.4.1 重新枚举：

新增 test_strategy2.cnf：

```
c U=1, I=2, S=3, C=4
p cnf 4 5
1 2 0
-3 1 0
-4 2 0
-3 -4 0
3 0
```

第一次运行的结果：

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat test_strategy2.cnf test_strategy2_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:          4
| Number of clauses:           4
| Parse time:                   0.00 s
| Eliminated clauses:           0.00 Mb
| Simplification time:          0.00 s
|
===== [ Search Statistics ] =====
| Conflicts | ORIGINAL | LEARNT | Progress |
|           | Vars  Clauses Literals | Limit  Clauses Lit/Cl |
=====
restarts      : 1
conflicts     : 0          (0 /sec)
decisions     : 1          (0.00 % random) (273 /sec)
propagations  : 3          (819 /sec)
conflict literals : 0          (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.003664 s

SATISFIABLE
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat test_strategy2_result.txt
SAT
1 -2 3 -4 0
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$
```

构造阻塞子句：-1 2 -3 4 0，加入 test_strategy2.cnf 最后一行；

同时把 p cnf 4 5 改为 p cnf 4 6 。

第二次运行的结果：

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat test_strategy2.cnf test_strategy2_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:          4
| Number of clauses:           4
| Parse time:                   0.00 s
| Simplification time:          0.00 s
|
|===== [ Search Statistics ] =====
| Conflicts | ORIGINAL | LEARNT | Progress |
|           | Vars  Clauses Literals | Limit  Clauses Lit/C1 |
|=====
restarts      : 1
conflicts     : 0          (0 /sec)
decisions     : 1          (0.00 % random) (223 /sec)
propagations  : 4          (893 /sec)
conflict literals : 0          (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.004477 s

SATISFIABLE
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat test_strategy2_result.txt
SAT
1 2 3 -4 0
```

构造阻塞子句：-1 -2 -3 4 0，加入 test_strategy2.cnf 最后一行；

同时把 p cnf 4 6 改为 p cnf 4 7 。

第三次运行的结果

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat test_strategy2.cnf test_strategy2_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:          4
| Number of clauses:           4
| Parse time:                   0.00 s
| Simplification time:          0.00 s
|
|=====
Solved by simplification
restarts      : 0
conflicts     : 0          (0 /sec)
decisions     : 0          (-nan % random) (0 /sec)
propagations  : 4          (795 /sec)
conflict literals : 0          (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.005031 s

UNSATISFIABLE
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat test_strategy2_result.txt
UNSAT
```

这次的结果变成了 UNSAT 。

5.4.2 比较修改前后的方案数量，并解释哪些方案被新规则排除：

PS:我注意到这里的 number of clauses 一直为4，没有像5.2那样随着阻塞子句的增加而增加。在询问了LLM之后得知，这是因为 minisAT 在读到 3 0 时触发了单元传播，直接固定了三个变量：S=true, U=true, C=false，接下来只要记住I即可，因此，后面读入的阻塞子句立即被化简为单元赋值或冲突，不必作为普通子句保存。

而之前加了是因为没有提前确定的变量值，不能直接化简为某个变量必须为真，所以 MinisAT 就把整条子句保存下来，内部计数加1。

对比如下：（未被删除线划掉的方案为加入 3 0 后允许的方案）

运行编号	运行结果	对应的测试策略	最后加入的阻塞子句
1	SAT, 1 2 3 4 0	只执行单元测试	1 2 3 4 0
2	SAT, 1 2 3 4 0	只执行集成测试	1 2 3 4 0
3	SAT, 1 2 3 4 0	同时执行单元测试与集成测试	1 2 3 4 0
4	SAT, 1 2 3 4 0	同时执行单元测试、集成测试与静态分析	1 2 3 4 0
5	SAT, 1 2 3 4 0	同时执行单元测试与静态分析	1 2 3 4 0
6	SAT, 1 2 3 4 0	执行集成测试的同时收集覆盖率	1 2 3 4 0
7	SAT, 1 2 3 4 0	同时执行单元测试与集成测试并收集覆盖率	1 2 3 4 0
8	UNSAT	无	无

对比修改前的方案，之前五个未执行静态分析的方案均被排除（1，2，3，6，7）。

保留的两个方案均执行单元测试和静态分析，不收集覆盖率，区别仅在于是否执行集成测试。

对阻塞子句为什么能够逐个排除已有完整赋值自行撰写的结论：

阻塞子句要求下一次的赋值至少有一个变量与当前模型不同。构造时只排除当前模型，不排除其他答案：

- 将当前完整模型中的每个文字反号，再用或连接。当前模型会使这些文字全部为假，因此无法满足这条子句；
- 但是其他完整赋值因为至少有一个变量不同，所以能使其中一个文字为真。所以，一条阻塞子句恰好排除一个完整赋值。

所以，每次找到新模型后继续添加对应的阻塞子句，并保留之前的阻塞子句，就能逐个排除已有模型，避免重复找到相同方案。

任务 6：检查两份 CNF 是否等价

任务摘要： 系统检查两份写法不同的候选 CNF 是否具有相同模型集合，解释它们为何等价，再通过删除一个子句制造差异并寻找区分赋值。

某服务包含以下六个业务变量：

变量	编号	为真时的含义
A	1	启用 API 服务
W	2	启用 Web 界面
D	3	启用数据库
C	4	启用缓存
L	5	启用日志
M	6	启用监控

需求如下：

1. API 服务和 Web 界面至少启用一个。
2. API 服务启用当且仅当数据库启用。
3. Web 界面启用时必须启用缓存。
4. 缓存启用时必须启用数据库。
5. 数据库启用时必须启用日志。
6. 监控启用当且仅当日志启用。

下面是两份候选编码。它们的子句数量和具体形式不同，不要仅根据文件外观判断二者是否等价。保存第一份为`encoding\_a.cnf`：

```
c A=1, W=2, D=3, C=4, L=5, M=6
p cnf 6 8
1 2 0
-1 3 0
-3 1 0
-2 4 0
-4 3 0
-3 5 0
-6 5 0
-5 6 0
```

保存第二份为 encoding_b.cnf：

```
c A=1, W=2, D=3, C=4, L=5, M=6
p cnf 6 12
-4 3 0
-3 5 0
-6 5 0
-5 6 0
1 2 4 0
1 2 -4 0
-1 3 6 0
-1 3 -6 0
-3 1 2 0
-3 1 -2 0
-2 4 5 0
-2 4 -5 0
```

两个 CNF 在业务变量上等价，是指对 A、W、D、C、L、M 的每一种完整赋值，两份 CNF 的满足结果都相同。能够满足其中一份而不满足另一份的赋值称为 **区分赋值** (distinguishing assignment)。如果不存在区分赋值，两个 CNF 的模型集合相同。

6.1. 把六条需求写成命题逻辑公式，并分别运行两份编码，记录求解状态和一个模型。(两份编码都是 SAT 只能说明它们各自至少有一个模型，不能据此判断模型集合是否相同；等价性由下一步的完整赋值检查确定。)

编号	需求	命题逻辑公式
1	API 服务和 Web 界面至少启用一个	$A \vee W$
2	API 服务启用当且仅当数据库启用	$A \leftrightarrow D$
3	Web 界面启用时必须启用缓存	$W \rightarrow C$
4	缓存启用时必须启用数据库	$C \rightarrow D$
5	数据库启用时必须启用日志	$D \rightarrow L$
6	监控启用当且仅当日志启用	$M \leftrightarrow L$

运行 encoding_a.cnf：

```

jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat encoding_a.cnf a_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:          6
| Number of clauses:           8
| Parse time:                  0.00 s
| Eliminated clauses:          0.00 Mb
| Simplification time:         0.00 s
|
===== [ Search Statistics ] =====
| Conflicts |          ORIGINAL          |          LEARNT          | Progress |
|           |  Vars  Clauses Literals  |  Limit  Clauses Lit/Cl   |          |
=====
restarts      : 1
conflicts     : 0              (0 /sec)
decisions     : 1              (0.00 % random) (110 /sec)
propagations  : 2              (220 /sec)
conflict literals : 0          (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.0091 s

SATISFIABLE
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat a_result.txt
SAT
1 -2 3 -4 5 6 0

```

运行 encoding_b.cnf :

```
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ minisat encoding_b.cnf b_result.txt
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:          6
| Number of clauses:           12
| Parse time:                   0.00 s
| Eliminated clauses:           0.00 Mb
| Simplification time:          0.00 s
|
===== [ Search Statistics ] =====
| Conflicts |          ORIGINAL          |          LEARNT          | Progress |
|           | Vars  Clauses Literals  |   Limit  Clauses Lit/Cl |          |
=====
restarts      : 1
conflicts     : 0              (0 /sec)
decisions     : 1              (0.00 % random) (3788 /sec)
propagations  : 2              (7576 /sec)
conflict literals : 0          (-nan % deleted)
Memory used   : 11.00 MB
CPU time      : 0.000264 s

SATISFIABLE
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ cat b_result.txt
SAT
1 -2 3 -4 5 6 0
```

可以看到，求解状态均为 SAT，模型是 1 -2 3 -4 5 6 0。

即：启用API服务，数据库，日志和监控。

6.2. 设计一个检查函数，枚举六个变量的全部 $2^6=64$ 种赋值，并调用任务 4 的 satisfiesCnf 比较 A 和 B 在每种赋值下的结果，从而判断两个编码是否等价。函数的接口和结果记录方式由你决定。使用该函数检查两份原始编码，并在实验报告中提交函数代码、检查方法和结果。

新增一个 test_equivalence.cpp，内容如下：

```

#include<iostream>
#include<vector>
#include "lab2_solution.cpp"

bool check_equivalence(int variableCount,
    const std::vector<std::vector<int>>& clausesA,
    const std::vector<std::vector<int>>& clausesB){
    int modelCountA = 0;
    int modelCountB = 0;
    //记录有多少个区分赋值
    int differenceCount = 0;

    std::cout << std::boolalpha;

    //计算循环总数, 2^variableCount次
    int total = 1 << variableCount;
    //i可以表示0~2^variableCount-1,
    //转换成二进制的话可覆盖total种取值
    for(int i = 0; i < total; i++){
        std::vector<int> assignment(variableCount);
        for(int j = 0; j < variableCount; j++){
            //将读取的那一位移到最右边, 只保留最后一位
            assignment[j] = (i >> j) & 1;
        }
        bool resultA = satisfiesCnf(variableCount, clausesA, assignment);
        bool resultB = satisfiesCnf(variableCount, clausesB, assignment);

        if(resultA)
            modelCountA++;
        if(resultB)
            modelCountB++;
        if(resultA != resultB){
            differenceCount++;
            std::cout << "Distinguishing assignment (A W D C L M): ";
            for(int value : assignment)
                std::cout << value << ' ';
            std::cout << "| encoding A=" << resultA
                << " | encoding B=" << resultB << '\n';
        }
    }

    bool equivalent = differenceCount == 0;
    std::cout << "Assignments checked: " << total << '\n';
    std::cout << "Models of encoding A: " << modelCountA << '\n';
    std::cout << "Models of encoding B: " << modelCountB << '\n';
    std::cout << "Distinguishing assignments: " << differenceCount << '\n';
    std::cout << "Equivalent: " << equivalent << '\n';

    return equivalent;
}

int main(){
    const std::vector<std::vector<int>> clausesA = {
        {1, 2},
        {-1, 3},
        {-3, 1},
        {-2, 4},
        {-4, 3},
        {-3, 5},
        {-6, 5},
        {-5, 6}
    };
    const std::vector<std::vector<int>> clausesB = {
        {-4, 3},

```

```

        {-3, 5},
        {-6, 5},
        {-5, 6},
        {1, 2, 4},
        {1, 2, -4},
        {-1, 3, 6},
        {-1, 3, -6},
        {-3, 1, 2},
        {-3, 1, -2},
        {-2, 4, 5},
        {-2, 4, -5}
    };

    check_equivalence(6, clausesA, clausesB);

    return 0;
}

```

函数检查方法：

- 首先通过变量数计算出总共要枚举多少次赋值
- 然后循环，把 $0 \sim 2^{\text{variable}} - 1$ 分别看成六位二进制数存入 assignment 数组，每一位的0或1对应一个变量的假或真。
- 对每个 assignment 分别调用 satisfiesCnf()，检查编码A和B，若结果不同则记为区分赋值并打印
- 遍历结束后，区分赋值数为0即判定两份编码等价，否则不等价。

函数检查结果：

```

jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ g++ test_equivalence.cpp -o test_equivalence
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ ./test_equivalence
Assignments checked: 64
Models of encoding A: 3
Models of encoding B: 3
Distinguishing assignments: 0
Equivalent: true

```

6.3. 使用命题逻辑等价变换，逐步推导为什么下面的公式成立，并说明这项变换为什么改变了 CNF 的写法和子句数量，却没有改变模型集合：

K 等价于 $(K \text{ OR } x) \text{ AND } (K \text{ OR NOT } x)$

其中 K 可以是一个包含多个文字的子句， x 是另一个布尔变量。推导中应写明所使用的分配律、矛盾律和同一律。

证： $(K \text{ OR } x) \text{ AND } (K \text{ OR NOT } x)$
 $\equiv (K \vee x) \wedge (K \vee \neg x)$ (转换成逻辑公式)
 $\equiv K \vee (x \wedge \neg x)$ (分配律)
 $\equiv K \vee \text{false}$ (矛盾律)
 $\equiv K$ (同一律)

整个公式的真假取决于K的真假，不论 x 如何取值，右边的真假始终与 K 相同：

- 若K为真，原式可写作： $(\text{true} \vee x) \wedge (\text{true} \vee \neg x) \equiv \text{true} \wedge \text{true} \equiv \text{true}$
- 若K为假，原式可写作： $(\text{false} \vee x) \wedge (\text{false} \vee \neg x) \equiv x \wedge \neg x \equiv \text{false}$

所以，虽然写法和子句数量发生变化，但对每一种完整赋值，替换前后的真假始终一致，所以模型集合不变。

6.4. 删除 B 中的子句 1 2 4 0 并把子句数改为 11。使用任意可靠方法找出一条使原始编码 A 与修改后的编码 B 得到不同结果的赋值。列出 A、W、D、C、L、M 的真假，将其翻译为完整的服务配置；再把该赋值分别代入两个编码，指出结果不同的具体子句和对应的业务需求。

6.4.1 使用任意可靠方法找出一条使原始编码 A 与修改后的编码 B 得到不同结果的赋值：

- 先新增一个文件 encoding_b2.cnf :删除B中的子句 1 2 4 0 并把子句数改为11。
- 然后，可以使用我在6.2里设计的检查函数，把 test_equivalence.cpp 中clausesB的 {1, 2, 4}，这一行删去：

```

63     const std::vector<std::vector<int>> clausesB = {
64         {-4, 3},
65         {-3, 5},
66         {-6, 5},
67         {-5, 6},
68         {1, 2, -4},
69         {-1, 3, 6},
70         {-1, 3, -6},
71         {-3, 1, 2},
72         {-3, 1, -2},
73         {-2, 4, 5},
74         {-2, 4, -5}
75     };

```

- 运行 test_equivalence.cpp，查看结果：

```

jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ g++ test_equivalence.cpp -o test_equivalence
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ ./test_equivalence
Distinguishing assignment (A W D C L M): 0 0 0 0 0 0 | encoding A=false | encoding B=true
Distinguishing assignment (A W D C L M): 0 0 0 0 1 1 | encoding A=false | encoding B=true
Assignments checked: 64
Models of encoding A: 3
Models of encoding B: 5
Distinguishing assignments: 2
Equivalent: false

```

这里和6.2一开始出现了错误结果：

```

jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ g++ test_equivalence.cpp -o test_equivalence
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ ./test_equivalence
Assignments checked: 64
Models of encoding A: 24347
Models of encoding B: 1721487587
Distinguishing assignments: 0
Equivalent: true
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ g++ test_equivalence.cpp -o test_equivalence
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$ ./test_equivalence
Assignments checked: 64
Models of encoding A: 23537
Models of encoding B: -1542069677
Distinguishing assignments: 0
Equivalent: true
jie@Saviour:/mnt/d/ME/FormalMethods/lab2$

```

经检查后发现，这是因为我在一开始忘了给 modelCountA 与 modelCountB 初始化。

现二者均已修订：

```

const std::vector<std::vector<int>> & clauses
{
    int modelCountA = 0;
    int modelCountB = 0;
}

```

最后将 {1, 2, 4}，补充回cpp文件中。

6.4.2 列出 A、W、D、C、L、M 的真假，将其翻译为完整的服务配置：

```

Distinguishing assignment (A W D C L M): 0 0 0 0 0 0 | encoding A=false | encoding B=true
Distinguishing assignment (A W D C L M): 0 0 0 0 1 1 | encoding A=false | encoding B=true

```

变量	功能	区分赋值 1	区分赋值 2
A	API 服务	false, 关闭	false, 关闭
W	Web 界面	false, 关闭	false, 关闭
D	数据库	false, 关闭	false, 关闭
C	缓存	false, 关闭	false, 关闭
L	日志	false, 关闭	true, 启用
M	监控	false, 关闭	true, 启用

即：

- (区分赋值1): API服务，Web界面，数据库，缓存，日志，监控均不启用。
- (区分赋值2): 只启用日志与监控。

6.4.3 把赋值分别代入两个编码，指出结果不同的具体子句和对应的业务需求：

以区分赋值1为例：(A W D C L M): F F F F F F

- A:

子句	代入结果
1 2 0	$F \vee F = F$
-1 3 0	$T \vee F = T$
-3 1 0	$T \vee F = T$
-2 4 0	$T \vee F = T$
-4 3 0	$T \vee F = T$
-3 5 0	$T \vee F = T$
-6 5 0	$T \vee F = T$
-5 6 0	$T \vee F = T$

因为A不满足第一条 1 2 0，所以最终结果为 false。1 2 0 对应的是业务需求1：API 服务和 Web 界面至少启用一个。

- B:

子句	代入结果
-4 3 0	$T \vee F = T$
-3 5 0	$T \vee F = T$
-6 5 0	$T \vee F = T$
-5 6 0	$T \vee F = T$
1 2 -4 0	$F \vee F \vee T = T$
-1 3 6 0	$T \vee F \vee F = T$
-1 3 -6 0	$T \vee F \vee T = T$
-3 1 2 0	$T \vee F \vee F = T$
-3 1 -2 0	$T \vee F \vee T = T$
-2 4 5 0	$T \vee F \vee F = T$
-2 4 -5 0	$T \vee F \vee T = T$

B全部满足，均为 true。

对 语法形式不同为什么仍可能表达相同模型集合 撰写的结论：

因为判断两份编码是否等价，关键是看它们分别允许哪些赋值，而不是语法的形式。就如6.3一样，一条子句被替换成两条与之等价的子句，但对于不同的赋值二者最后的真假都相同。

LLM使用说明:

1. 通过LLM了解到:

PowerShell 找不到 minisat 程序，可能是尚未安装，或没有加入 PATH。
Windows 下建议通过 WSL + Ubuntu 安装和运行。

2. 通过LLM了解DIMACS CNF相关的规则和变量说明。

3. 通过LLM获悉md文件逻辑符号如何表示: (*LaTeX*)

$\backslash($ 和 $\backslash)$: 标记一段行内公式的开始和结束，正常渲染后不会显示。

普通的 (和): 公式内容中的括号，会显示出来。

4. 使用LLM为自选的软件工程场景生成SAT模型，整理自然语言需求与CNF子句的对应关系，再使用MiniSAT求解。(任务3)

5. 使用LLM辅助完成任务4，任务5与任务6的代码理解与编程。

6. 利用LLM解释 number of clauses 的在任务5不同场景变化不同的原因。